

Lumina CV Studio: Reference Report

This reference document lists the mathematical operations and implementation logic for spatial enhancements and geometrical transformations supported in Lumina CV Studio.

Spatial-Domain Enhancement

1. Brightness Adjustment

Formula: $s = r + \beta$ (where $\beta = 50$)

Description: Adds a constant brightness offset to each pixel intensity. Pixel values are clipped to the valid range [0, 255] to prevent overflow wraps.

```
beta = 50
img_proc = np.clip(img_orig.astype(np.int32) + beta, 0, 255).astype(np.uint8)
```

Original (Input) Image



Processed (Output) Image



2. Contrast Adjustment

Formula: $s = \alpha \cdot r$ (where $\alpha = 1.6$)

Description: Scales pixel intensities by a constant gain factor. Values greater than 1 stretch the image contrast, while values less than 1 compress it.

```
alpha = 1.6
img_proc = np.clip(img_orig.astype(np.float32) * alpha, 0, 255).astype(np.uint8)
```

Original (Input) Image



Processed (Output) Image



3. Image Negative

Formula: $s = 255 - r$

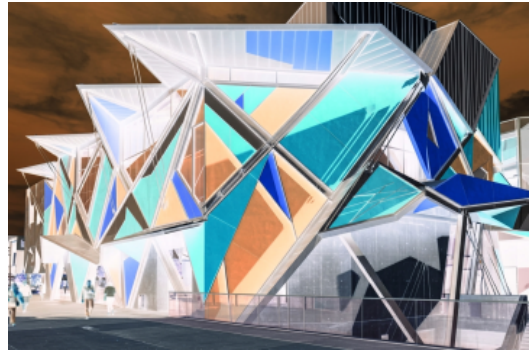
Description: Inverts the intensity levels of the image, transforming light pixels into dark and vice-versa, making structural details in dark regions easier to analyze.

```
img_proc = (255 - img_orig).astype(np.uint8)
```

Original (Input) Image



Processed (Output) Image



4. Log Transformation

Formula: $s = c \cdot \ln(1 + r)$

Description: Compresses the dynamic range of images containing extremely high intensity variations. It expands low-intensity (dark) values and compresses high-intensity (bright) values.

```
c_factor = 1.2  
r = img_orig.astype(np.float32)  
c_base = 255.0 / np.log(1.0 + np.max(r)) if np.max(r) > 0 else 1.0  
img_proc = np.clip(c_base * c_factor * np.log(1.0 + r), 0, 255).astype(np.uint8)
```

Original (Input) Image



Processed (Output) Image



5. Power-Law (Gamma) Transformation

Formula: $s = c \cdot r^\gamma$ (where $\gamma = 0.4$, $c = 1.0$)

Description: Applies non-linear correction to mid-tones. A fractional gamma ($\gamma < 1$) maps a narrower range of dark inputs into a wider range of outputs (brightens), while $\gamma > 1$ compresses dark tones (darkens).

```
gamma = 0.4
c_factor = 1.0
r = img_orig.astype(np.float32) / 255.0
img_proc = np.clip(255.0 * c_factor * (r ** gamma), 0, 255).astype(np.uint8)
```

Original (Input) Image



Processed (Output) Image



6. Thresholding (Grayscale)

Formula: $s = 255$ if $r_{gray} \geq T$ else 0 (where $T = 120$)

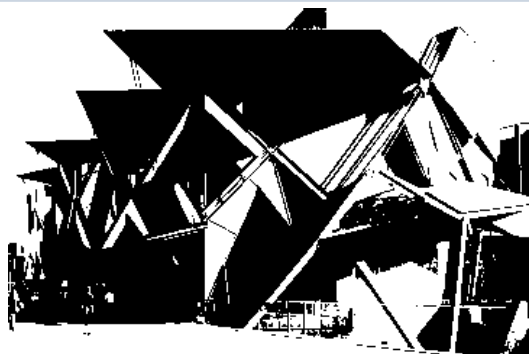
Description: Converts a grayscale version of the image into a binary black-and-white output. Intensity values greater than or equal to the threshold value T are set to 255 (white), others to 0 (black).

```
threshold_val = 120
gray = cv2.cvtColor(img_orig, cv2.COLOR_RGB2GRAY)
_, thresh = cv2.threshold(gray, threshold_val, 255, cv2.THRESH_BINARY)
img_proc = cv2.cvtColor(thresh, cv2.COLOR_GRAY2RGB)
```

Original (Input) Image



Processed (Output) Image



7. Thresholding (RGB Channel-wise)

Formula: $s_i = 255$ if $r_i \geq T$ else 0 ($T = 120$ for RGB)

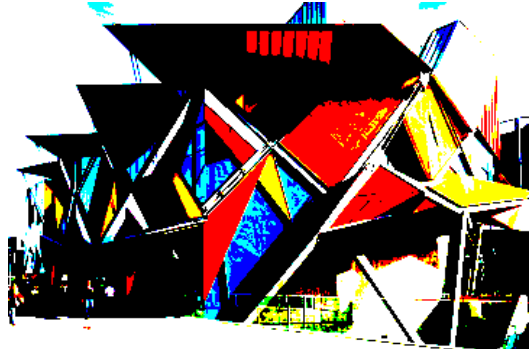
Description: Performs binary thresholding independently on each color channel. This results in vibrant, highly saturated primary/secondary colors.

```
threshold_val = 120
img_proc = np.where(img_orig >= threshold_val, 255, 0).astype(np.uint8)
```

Original (Input) Image



Processed (Output) Image



8. Contrast Stretching

Formula: $s = (r - r_{min}) \cdot (s_{max} - s_{min}) / (r_{max} - r_{min}) + s_{min}$

Description: Linearly maps narrow input intensity range $[r_{min}, r_{max}]$ to fill the entire dynamic destination range $[s_{min}, s_{max}]$ (usually $[0, 255]$). Useful for brightening low-contrast, washed-out images.

```
r_min, r_max = int(np.min(img_orig)), int(np.max(img_orig))
s_min, s_max = 0, 255
r = img_orig.astype(np.float32)
denom = (r_max - r_min) if r_max != r_min else 1.0
stretched = (r - r_min) * ((s_max - s_min) / denom) + s_min
img_proc = np.clip(stretched, 0, 255).astype(np.uint8)
```

Original (Input) Image



Processed (Output) Image



Geometrical Transformations

9. Translation

Formula: $x' = x + t_x$, $y' = y + t_y$ (where $t_x = 50$, $t_y = 30$)

Description: Shifts the image spatial coordinates horizontally and vertically by specified pixel displacements. Empty boundary regions are filled with a default background color.

```
tx, ty = 50, 30
rows, cols = img_orig.shape[:2]
M = np.float32([[1, 0, tx], [0, 1, ty]])
img_proc = cv2.warpAffine(img_orig, M, (cols, rows), borderValue=(30, 41, 59))
```

Original (Input) Image



Processed (Output) Image



10. Scaling (Bilinear)

Formula: $x' = s_x \cdot x$, $y' = s_y \cdot y$ (where $s_x = 1.3$, $s_y = 1.3$)

Description: Resizes the spatial footprint of the image. Standard bilinear interpolation is used to estimate intermediate pixel intensities, producing a smooth scaling result.

```
sx, sy = 1.3, 1.3
rows, cols = img_orig.shape[:2]
new_w, new_h = int(cols * sx), int(rows * sy)
img_proc = cv2.resize(img_orig, (new_w, new_h), interpolation=cv2.INTER_LINEAR)
```

Original (Input) Image



Processed (Output) Image



11. Rotation

Formula: Rotate by θ around pivot (x_c, y_c) ($\theta = 25^\circ$, $scale = 0.95$)

Description: Rotates the coordinates by an angle theta around a central pivot point, optionally scaling the image dimensions relative to the center to keep it in frame.

```
angle, scale = 25.0, 0.95
rows, cols = img_orig.shape[:2]
xc, yc = cols / 2.0, rows / 2.0
M = cv2.getRotationMatrix2D((xc, yc), angle, scale)
img_proc = cv2.warpAffine(img_orig, M, (cols, rows), borderValue=(30, 41, 59))
```

Original (Input) Image



Processed (Output) Image



12. Horizontal Reflection

Formula: $x' = W - 1 - x$, $y' = y$

Description: Flips the image along the vertical axis of symmetry (mirror effect), reversing the left and right layout of the image details.

```
img_proc = cv2.flip(img_orig, 1)
```

Original (Input) Image



Processed (Output) Image



13. Vertical Reflection

Formula: $x' = x, y' = H - 1 - y$

Description: Flips the image upside down along the horizontal axis of symmetry.

```
img_proc = cv2.flip(img_orig, 0)
```

Original (Input) Image



Processed (Output) Image



14. Reflection about Origin

Formula: $x' = W - 1 - x, y' = H - 1 - y$ (180° rotation)

Description: Flips coordinates across both the horizontal and vertical axes of symmetry, equivalent to rotating the image by 180 degrees around its center.

```
img_proc = cv2.flip(img_orig, -1)
```

Original (Input) Image



Processed (Output) Image



15. Shearing (X-Axis)

Formula: $x' = x + k_x \cdot y$, $y' = y$ (where $k_x = 0.25$)

Description: Displaces the horizontal coordinate of each pixel proportional to its vertical distance from the origin. This slants the image shapes sideways.

```
kx = 0.25
rows, cols = img_orig.shape[:2]
M = np.float32([[1, kx, 0], [0, 1, 0]])
img_proc = cv2.warpAffine(img_orig, M, (cols, rows), borderValue=(30, 41, 59))
```

Original (Input) Image



Processed (Output) Image



16. Shearing (Y-Axis)

Formula: $x' = x$, $y' = y + k_y \cdot x$ (where $k_y = 0.25$)

Description: Displaces the vertical coordinate of each pixel proportional to its horizontal distance from the origin. This slants the image shapes vertically.

```
ky = 0.25
rows, cols = img_orig.shape[:2]
M = np.float32([[1, 0, 0], [ky, 1, 0]])
img_proc = cv2.warpAffine(img_orig, M, (cols, rows), borderValue=(30, 41, 59))
```

Original (Input) Image



Processed (Output) Image



17. Affine Transformation

Formula: Warp using 3 non-collinear matching control points

Description: Calculates a 2x3 transformation matrix from three matching control points. Preserves collinearity and straight lines, but alters angles, distances, and shapes.

```
dx1, dy1 = 20, 15
dx2, dy2 = -30, 40
dx3, dy3 = 30, -20
rows, cols = img_orig.shape[:2]
src_pts = np.float32([[0, 0], [cols-1, 0], [0, rows-1]])
dst_pts = np.float32([
    [0 + dx1, 0 + dy1],
    [cols-1 + dx2, dy2],
    [dx3, rows-1 + dy3]
])
M = cv2.getAffineTransform(src_pts, dst_pts)
img_proc = cv2.warpAffine(img_orig, M, (cols, rows), borderValue=(30, 41, 59))
```

Original (Input) Image



Processed (Output) Image



Complete Reference Source Code

The following blocks compile the full standalone reference code for Lumina CV Studio's image manipulation algorithms. The script is free of UI/UX code and can be run independently.

1. Setup & Utility Imports

```
import cv2
import numpy as np
```

2. Spatial Domain Point Processing Helpers

```
def adjust_brightness(img, beta=50):
    """Adds a constant brightness offset to each pixel intensity.
    Values are clipped to the valid range [0, 255] to prevent overflow.
    """
    return np.clip(img.astype(np.int32) + beta, 0, 255).astype(np.uint8)

def adjust_contrast(img, alpha=1.6):
    """Scales pixel intensities by a constant gain factor.
    alpha > 1 stretches contrast; alpha < 1 compresses it.
    """
    return np.clip(img.astype(np.float32) * alpha, 0, 255).astype(np.uint8)

def image_negative(img):
    """Inverts the intensity levels of the image (s = 255 - r)."""
    return (255 - img).astype(np.uint8)

def log_transformation(img, c_factor=1.2):
    """Compresses dynamic range by expanding dark intensities
    and compressing bright intensities.
    """
    r = img.astype(np.float32)
    c_base = 255.0 / np.log(1.0 + np.max(r)) if np.max(r) > 0 else 1.0
    return np.clip(c_base * c_factor * np.log(1.0 + r), 0, 255).astype(np.uint8)
```

3. Advanced Spatial Enhancements

```
def gamma_transformation(img, gamma=0.4, c=1.0):
    """Applies power-law intensity mapping.
    gamma < 1 brightens mid-tones; gamma > 1 darkens mid-tones.
    """
    r = img.astype(np.float32) / 255.0
    return np.clip(255.0 * c * (r ** gamma), 0, 255).astype(np.uint8)

def threshold_grayscale(img, threshold_val=120):
    """Converts a grayscale version of the image into binary black-and-white."""
    gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
    _, thresh = cv2.threshold(gray, threshold_val, 255, cv2.THRESH_BINARY)
    return cv2.cvtColor(thresh, cv2.COLOR_GRAY2RGB)

def threshold_rgb(img, threshold_val=120):
    """Performs binary thresholding independently on each color channel."""
    return np.where(img >= threshold_val, 255, 0).astype(np.uint8)

def contrast_stretching(img, s_min=0, s_max=255):
    """Linearly expands the narrow input dynamic range to fill [s_min, s_max]."""
    r_min, r_max = int(np.min(img)), int(np.max(img))
    r = img.astype(np.float32)
    denom = (r_max - r_min) if r_max != r_min else 1.0
    stretched = (r - r_min) * ((s_max - s_min) / denom) + s_min
    return np.clip(stretched, 0, 255).astype(np.uint8)
```

4. Geometrical Transformations (Affine & Scaling)

```
def translate_image(img, tx=50, ty=30, bg_color=(30, 41, 59)):
    """Shifts image spatial coordinates by tx horizontally and ty vertically."""
    h, w = img.shape[:2]
    M = np.float32([[1, 0, tx], [0, 1, ty]])
    return cv2.warpAffine(img, M, (w, h), borderValue=bg_color)

def scale_image(img, sx=1.3, sy=1.3):
    """Resizes the width and height of the image using bilinear interpolation."""
    h, w = img.shape[:2]
    new_w, new_h = int(w * sx), int(h * sy)
    return cv2.resize(img, (new_w, new_h), interpolation=cv2.INTER_LINEAR)

def rotate_image(img, angle=25.0, scale=0.95, bg_color=(30, 41, 59)):
    """Rotates the image by angle degrees around its center point."""
    h, w = img.shape[:2]
    xc, yc = w / 2.0, h / 2.0
    M = cv2.getRotationMatrix2D((xc, yc), angle, scale)
    return cv2.warpAffine(img, M, (w, h), borderValue=bg_color)
```

5. Reflections & Shearing

```
def reflect_horizontal(img):
    """Flips the image horizontally (mirror flip along vertical axis)."""
    return cv2.flip(img, 1)

def reflect_vertical(img):
    """Flips the image vertically (upside-down flip along horizontal axis)."""
    return cv2.flip(img, 0)

def reflect_origin(img):
    """Flips the image coordinates across both axes (180 degree rotation)."""
    return cv2.flip(img, -1)

def shear_x(img, kx=0.25, bg_color=(30, 41, 59)):
    """Applies horizontal shear by factor kx."""
    h, w = img.shape[:2]
    M = np.float32([[1, kx, 0], [0, 1, 0]])
    return cv2.warpAffine(img, M, (w, h), borderValue=bg_color)

def shear_y(img, ky=0.25, bg_color=(30, 41, 59)):
    """Applies vertical shear by factor ky."""
    h, w = img.shape[:2]
    M = np.float32([[1, 0, 0], [ky, 1, 0]])
    return cv2.warpAffine(img, M, (w, h), borderValue=bg_color)
```

6. Affine Warp & Standalone Main Execution Block

```
def affine_transform(img, dx1=20, dy1=15, dx2=-30, dy2=40, dx3=30, dy3=-20, bg_color=(30, 41, 59)):
    """Warp the image matching 3 control points (Top-Left, Top-Right, Bottom-Left)."""
    h, w = img.shape[:2]
    src_pts = np.float32([[0, 0], [w-1, 0], [0, h-1]])
    dst_pts = np.float32([
        [0 + dx1, 0 + dy1],
        [w-1 + dx2, dy2],
        [dx3, h-1 + dy3]
    ])
    M = cv2.getAffineTransform(src_pts, dst_pts)
    return cv2.warpAffine(img, M, (w, h), borderValue=bg_color)

if __name__ == "__main__":
    # Standalone execution test with test image loading
    img_bgr = cv2.imread("default_input.jpg")
    if img_bgr is None:
        print("default_input.jpg not found. Creating dummy test image.")
        img_rgb = np.zeros((300, 450, 3), dtype=np.uint8)
        cv2.circle(img_rgb, (225, 150), 80, (255, 0, 0), -1)
    else:
        img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

    print(f"Loaded source image shape: {img_rgb.shape}")
    brightened = adjust_brightness(img_rgb, beta=50)
    rotated = rotate_image(img_rgb, angle=30)
    print(f"Processed brightened shape: {brightened.shape}")
    print(f"Processed rotated shape: {rotated.shape}")
```